

Lecture 3 List, Stack and Queue

Data Types

The set of **objects** and their **corresponding operations**.

Abstract data type(ADT) is data type **defined by users**.

List

- A general list of the form $a_1, a_2, \dots, a_n$.
- The size of the list is n .
- Null list: a list of size 0.

The List ADT

- Object: $a_1, a_2, \dots, a_n$.
- Operations: Length, insert, delete and so on.

Array Implementation(Sequential List)

Common definition of data type of sequential list:

```
#define MAXLEN 100

typedef struct{
    ElemType data[MAXLEN];
    int Length;
}SeqList;
```

Operations:

- Initialize a sequential list
- Insertion
- Deletion
- Find k -th element

Linked List

A series of connected structures.

- A piece of data

- A pointer to the next structure in the list.
The next of last structure points to the NULL.

```
struct Node;
typedef struct Node *node_ptr;

struct Node{
    ElemType data;
    node_ptr next;
};

typedef node_ptr List;
typedef node_ptr position;
```

The linked list may have a dummy head node.

Operations:

- Is_Empty
- Is_last
- Insert and deletion

A singly linked **circular list** with the head node: the next of the last node point to the head node.

Doubly Linked List

similar to linked list but two pointers. One to the previous node and another to the next node.

```
typedef struct Dlnode* Dlnode_ptr;
struct Dlnode{
    ElemType data;
    Dlnode_ptr prior, next;
};
```

Operations:

- Insertion
- Deletion

For a doubly linked **circular list** with the head node: the next of the last node point to the head node. The prior of the head node is the last node.

Example: The Polynomial ADT and Operations:

- Addition
- Subtraction
- Multiplication
- Differentiation

Stack

Definition

An ordered group of homogeneous items or elements. Add and remove element at the **top** of the stack. (Last-in-First-out LIFO)

ADT

Objects: A finite ordered list with elements

Operations:

- Push
- Pop and return the top element
- Is empty
- Create stack
- Destroy stack
- Make empty
- Return the top without pop

Implemented by linked list (more common) or array.

In PPT it has a **dummy head node** for linked list implementation.

Application I

Check if the braces are balanced.

Application II

Evaluating Expressions.

Queue

Definition

An ordered collection of items which insertion takes place at one end (**rear**) and deletion takes place at the opposite end (**front**). (FIFO)

ADT

Objects: A finite ordered list with elements

Operations:

- Enqueue
- Dequeue and return the front element
- Is empty
- Create Queue
- Destroy Queue
- Make Empty
- Return the front element and not dequeue.

Circular Queue

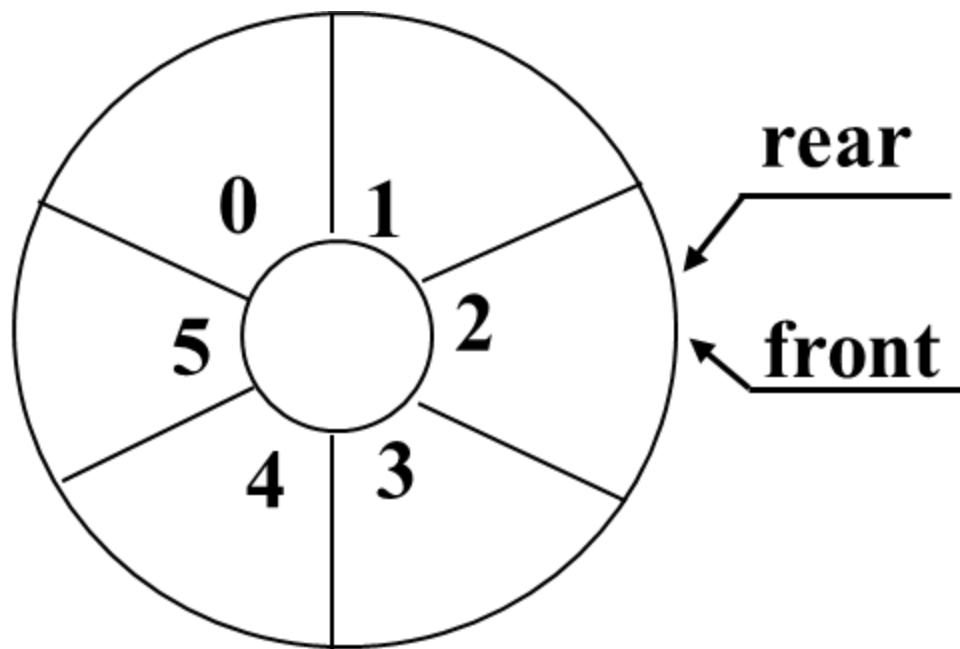
For enqueue, when $\text{rear} = \text{Capacity} - 1$, the next element is entered at $a[0]$ if it is free. ($\text{rear} = (\text{rear} + 1) \bmod \text{Capacity}$)

For dequeue, when $\text{front} = \text{Capacity} - 1$, the next element at $a[0]$ will be deleted if the queue is not empty. ($\text{front} = (\text{front} + 1) \bmod \text{Capacity}$)

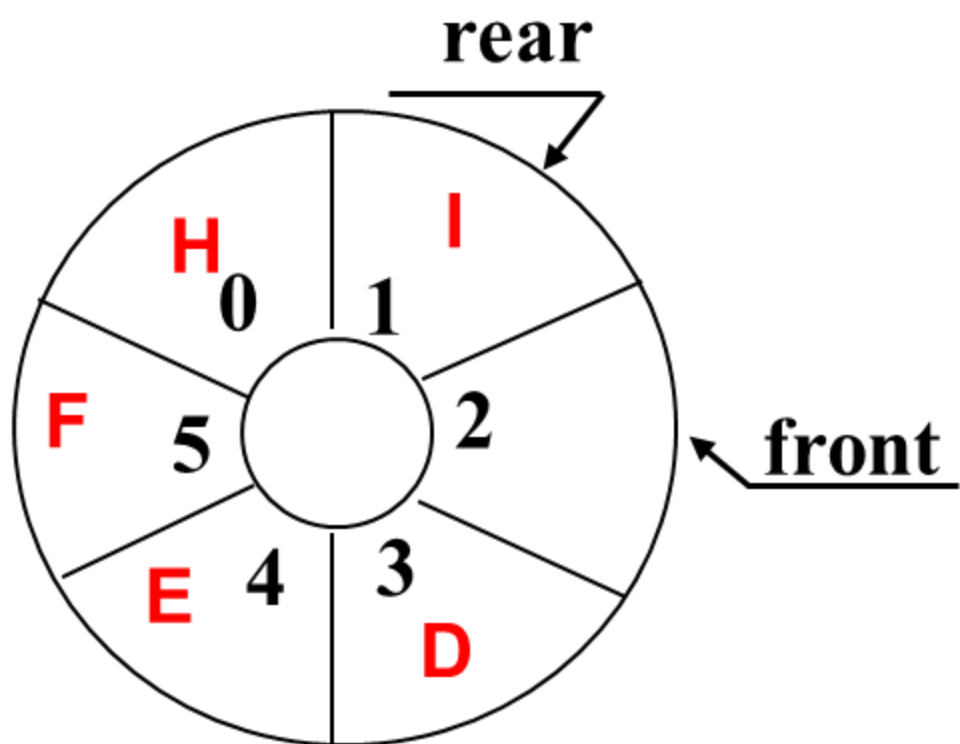
The related position between rear and front has $n - 1$ situation for array with size n . So it cannot express all situations and cannot judge whether it is empty or full. The solution is only use $n - 1$ size and regulate the front is always empty.

When the queue is empty, $\text{rear} = \text{front}$;

When the queue is full, $(\text{rear} + 1) \bmod \text{Capacity} = \text{front}$.



Empty circular queue



full circular queue

